Deepfake Image Classification: Technical Report

Summary of the dataset, EfficientNet-B0 training pipeline, validation metrics, corruption checks, and inference workflow.

1. Problem Statement & Dataset

The goal of this project is to classify face images as real (`0`) or deepfake (`1`). The classifier outputs a continuous confidence score in `[0, 1]` representing the predicted probability that an image is fake. Model performance is primarily evaluated using ROC AUC, complemented by accuracy and F1 score at a classification threshold of 0.5.

Dataset Breakdown

The dataset contains 7,000 labeled training images (3,478 real and 3,522 fake) and 3,000 unlabeled test images. All training image filenames align with the labels in `train_metadata.csv`, and all 3,000 test filenames match the target IDs in `predictions.csv`.

```
dataset/
  train/
    real/            # 3,478 images (label = 0)
    fake/            # 3,522 images (label = 1)
  test/images/       # 3,000 images
  train_metadata.csv
```

Validation Split

We evaluate on a stratified 20% validation split (`test_size=0.2`, `random_state=42`) comprising 1,400 images (696 real and 704 fake), leaving 5,600 images for training. Because `train_test_split` depends on input row ordering, maintaining a consistent file/metadata order ensures repeatable splits across runs.

2. Model Architecture & Training Pipeline

The classifier uses torchvision's EfficientNet-B0 initialized with ImageNet-pretrained weights (`IMAGENET1K_V1`). The default classification head is replaced with a dropout layer and a single-logit linear layer, giving 4,008,829 trainable parameters in total.

Component	Configuration
Backbone	EfficientNet-B0 (ImageNet-1K pretrained)
Regularization	Dropout (p = 0.3)
Classifier head	Linear (1,280 -> 1 logit)
Output probability	Sigmoid(logit) for fake class (label = 1)
Total parameters	4,008,829

Preprocessing & Augmentation

All images are converted to RGB, resized to 224 x 224 pixels, and normalized with standard ImageNet channel statistics (mean `[0.485, 0.456, 0.406]`, std `[0.229, 0.224, 0.225]`). During training, we apply random horizontal flipping, brightness/contrast jitter (0.1), and Gaussian blur (radius 0.5-1.5) with probability 0.2. Validation and test pipelines are deterministic.

Optimization Setup

Hyperparameter	Value
Loss function	BCEWithLogitsLoss
Optimizer	AdamW (lr = 3e-4, weight_decay = 1e-4)
Batch size / Epochs	32 / 10
Random seed	42 (Python, NumPy, PyTorch)
Mixed precision	CUDA autocast + GradScaler when GPU is available
Checkpoint criterion	Highest validation ROC AUC (saved to model_weights.pt)

In `training_log.txt`, training loss drops from 0.5270 at epoch 1 to 0.0811 at epoch 10, with the best checkpoint saved at epoch 8 (directory-ordered validation AUC of 0.8925). No learning-rate scheduler is used.

3. Validation & Robustness Results

To check both clean accuracy and sensitivity to common image degradations, the saved checkpoint (`model_weights.pt`) was evaluated on the 1,400-image stratified metadata split under clean conditions and three deterministic perturbations (JPEG re-encoding at quality 50, Gaussian blur with radius 1.0, and half-resolution bilinear downsampling followed by upscaling back before the 224 x 224 transform).

Validation Condition	ROC AUC	Accuracy	F1 Score
Clean	0.9861	0.9636	0.9634
JPEG quality 50	0.9822	0.9571	0.9573
Gaussian blur (radius 1)	0.9840	0.9557	0.9549
Half-resolution resize	0.9803	0.9457	0.9442

Across the three degradations, ROC AUC decreases modestly by 0.0021 to 0.0058 relative to the clean split, while thresholded accuracy drops most on half-resolution resizing (-0.0179). Evaluating the same checkpoint on a split built from class-directory iteration order gives a clean ROC AUC of 0.9823, compared with 0.9861 on the metadata-ordered split and 0.8925 in `training_log.txt`, reflecting differences in split ordering.

Runtime Observations

On CPU-only hardware (Python 3.10.11, PyTorch 2.14.1+cpu), evaluating the 1,400 validation images took 54.33 seconds (approximately 25.8 images/sec including loading and preprocessing). Inference uses batch size 64 and automatically enables CUDA mixed precision on GPU systems.

4. Inference Output, Limitations & Reproduction

Test Inference

Running `inference_script.py` over all 3,000 test images writes `predictions.csv` with columns `image_id,label,confidence`, binary predictions thresholded at 0.5, and sigmoid confidence scores bounded in `[0, 1]`.

Limitations

- Ground-truth test labels are unavailable locally, so test-set ROC AUC and accuracy cannot be directly measured.
- Robustness checks cover JPEG compression (Q=50), Gaussian blur (r=1), and 0.5x downsampling; other artifacts or unseen generators may behave differently.
- The divergence between the training log AUC (0.8925) and post-training split evaluations (0.9823 / 0.9861) highlights sensitivity to split ordering.
- Runtime measurements were recorded on CPU and will differ on GPU hardware.

Reproduction Commands

Install dependencies and run inference or training from the project root:

```
pip install -r requirements.txt

# Generate test predictions
python inference_script.py --test_dir dataset/test/images \
    --weights model_weights.pt --output predictions.csv

# Retrain from dataset root (expects ./train/real and ./train/fake)
cd dataset
python ../train.py
```